

# Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI  
I TECHNIK INFORMACYJNYCH



przedmiot  
ADAC - Analiza danych w cyberbezpieczeństwie



Mouse movement behavioral biometrics

Zespół FPDŁ, Semestr 26L, Projekt - raport

Dominik Łopatecki(CB),

Filip Przygoda(CB)

Numer albumu 310811, 311797

prowadzący  
mgr inż. Bartłomiej Mastej, dr Katarzyna Kamińska

WARSZAWA 20 maja 2026

## Spis treści

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| <b>1. Podział prac</b>                                                | 5  |
| <b>2. Cel i motywacja projektu</b>                                    | 5  |
| 2.1. Motywacja                                                        | 5  |
| 2.2. Cel i zakres projektu                                            | 6  |
| <b>3. Testbed i środowisko eksperymentalne</b>                        | 6  |
| 3.1. Architektura testbed'u                                           | 6  |
| 3.1.1. Komponenty systemu                                             | 7  |
| 3.1.2. Przepływ danych (Data Flow)                                    | 8  |
| 3.2. Wykorzystane technologie                                         | 8  |
| 3.3. Scenariusze eksperymentalne                                      | 10 |
| <b>4. Generacja i przygotowanie danych</b>                            | 10 |
| 4.1. Proces generacji i zbierania danych                              | 10 |
| 4.2. Opis zebranych danych                                            | 11 |
| 4.3. Analiza jakości danych                                           | 12 |
| <b>5. Feature engineering i preprocessing</b>                         | 12 |
| 5.1. Okna czasowe i agregacja danych                                  | 12 |
| 5.2. Opis wyekstrahowanych cech                                       | 12 |
| 5.3. Uzasadnienie wyboru cech                                         | 13 |
| 5.4. Transformacje danych (normalizacja i filtracja)                  | 13 |
| <b>6. Analiza, uczenie i ewaluacja modeli uczenia maszynowego</b>     | 14 |
| 6.1. Konfiguracja eksperymentów                                       | 14 |
| 6.2. Wybór i selekcja algorytmów klasyfikacyjnych                     | 15 |
| 6.2.1. Modele odrzucone i ich wyniki                                  | 15 |
| 6.2.1.1 Isolation Forest - metodyka i parametry modelu                | 15 |
| 6.2.1.2 Hybrydowa sieć CNN-LSTM - metodyka i architektura modelu      | 16 |
| 6.2.2. Wybrany model docelowy i jego wynik (Random Forest)            | 18 |
| 6.3. Eksperyment na gotowym środowisku                                | 20 |
| 6.4. Analiza wpływu przygotowania danych na efektywność klasyfikacji  | 21 |
| <b>7. Działanie aplikacji i scenariusz użytkownika</b>                | 21 |
| 7.0.1. Krok 1: Logowanie i wybór trybu pracy                          | 21 |
| 7.0.2. Krok 2: Tryb Zbierania Danych                                  | 22 |
| 7.0.3. Krok 3: Trenowanie modelu profilu                              | 22 |
| 7.0.4. Krok 4: Tryb Rozpoznawania (Weryfikacja w czasie rzeczywistym) | 23 |
| <b>8. Dyskusja i wnioski</b>                                          | 24 |
| 8.1. Materiały dodatkowe                                              | 25 |
| 8.1.1. Repozytorium projektu                                          | 25 |
| 8.1.2. Struktura projektu i opis plików                               | 25 |

|                                     |    |
|-------------------------------------|----|
| 8.1.3. Struktura datasetu . . . . . | 26 |
|-------------------------------------|----|



# 1. Podział prac

Tabela z oznaczeniem zadań zrealizowanych przez każdego członka zespołu.

**Tabela 1.1.** Wykaz zadań w projekcie i ich waga

| Nr | Zadanie                                       | %   |
|----|-----------------------------------------------|-----|
| 1  | Research problemu                             | 10% |
| 2  | Budowa strony webowej do zbierania danych     | 15% |
| 3  | Skonfig. bazy danych, implementacja backendu  | 25% |
| 4  | Ekstrakcja danych, budowa modeli i trenowanie | 20% |
| 5  | Ocena modeli, zebranie wyników, raport        | 20% |
| 6  | Dokumentacja                                  | 5%  |
| 7  | Prezentacja                                   | 5%  |

**Tabela 1.2.** Udział członków zespołu w realizacji projektu

| Osoba             | Zadania    | Udział |
|-------------------|------------|--------|
| Filip Przygoda    | 1, 2, 4, 7 | 50%    |
| Dominik Łopatecki | 3, 5, 6    | 50%    |

## 2. Cel i motywacja projektu

### 2.1. Motywacja

Główną motywacją projektu jest zaprojektowanie systemu ciągłej weryfikacji tożsamości użytkownika w oparciu o dynamikę ruchu myszy. W przeciwieństwie do klasycznych, punktowych metod uwierzytelniania (takich jak hasła czy jednorazowe kody 2FA), proponowane rozwiązanie działa w tle w sposób niewidoczny dla użytkownika. Pozwala to na nieprzerwane monitorowanie sesji i natychmiastowe wykrycie sytuacji przejęcia konta przez osobę nieuprawnioną lub zautomatyzowanego bota.

Podstawowym założeniem pracy jest hipoteza, że każdy człowiek wykazuje wysoce spersonalizowane cechy motoryczne podczas operowania urządzeniem wskazującym. Parametry takie jak dynamika ruchu, średnia prędkość, przyspieszenie, efektywność trajektorii czy precyzja najechania na cel, stanowią unikalną sygnaturę biometryczną. Ekstrakcja tych cech z surowych współrzędnych pozwala na zbudowanie indywidualnego profilu behawioralnego, który stanowi wzorzec referencyjny do porównywania z bieżącą aktywnością.

## 2.2. Cel i zakres projektu

Celem projektu jest implementacja funkcjonalnego prostego systemu weryfikacji tożsamości w czasie rzeczywistym oraz zbadanie skuteczności biometrii behawioralnej. Badanie ma na celu weryfikację, czy analiza ruchu myszy może stanowić wiarygodną, dodatkową warstwę zabezpieczeń, wspierającą proces uwierzytelniania w zaawansowanych systemach informatycznych.

W ramach części praktycznej przygotowano stanowisko badawcze w postaci aplikacji webowej – gry polegającej na precyzyjnym klikaniu w losowo pojawiające się obiekty. Środowisko to służy do bezinwazyjnego pozyskiwania strumieni surowych danych o ruchu kursora. Zgromadzony materiał poddawany jest procesowi inżynierii cech, a następnie służy do trenowania modeli uczenia maszynowego, których zadaniem jest klasyfikacja i potwierdzenie, czy osoba korzystająca z systemu jest prawowitym właścicielem konta.

Projekt obejmuje analizę i porównanie skuteczności wybranych modeli uczenia maszynowego, takich jak Random Forest, Isolation Forest oraz sieć CNN-LSTM, w kontekście:

- **Weryfikacji użytkownika:** określenia, czy aktualne zachowanie odpowiada właścicielowi konta,
- **Detekcji anomalii:** wykrywania nietypowych zachowań mogących wskazywać na przejęcie konta lub działanie automatycznego bota,
- **Oceny przydatności systemu jako dodatkowej warstwy bezpieczeństwa:** analizy skuteczności rozwiązania na podstawie wskaźników FAR (False Acceptance Rate) oraz FRR (False Rejection Rate),
- **Wyboru najlepszego modelu:** porównania skuteczności zastosowanych algorytmów i określenia rozwiązania osiągającego najlepsze wyniki.

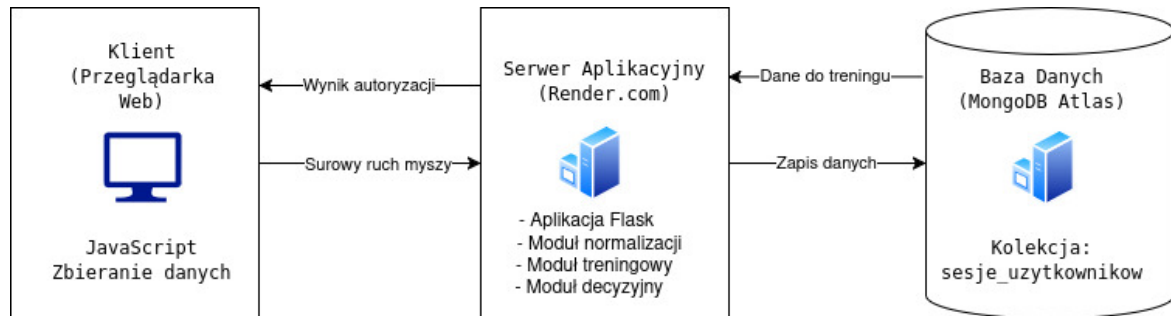
## 3. Testbed i środowisko eksperymentalne

Celem tego rozdziału jest zaprezentowanie stanowiska badawczego (ang. *testbed*), które zostało zaprojektowane i wdrożone na potrzeby ewaluacji systemu ciągłego uwierzytelniania behawioralnego. Środowisko to musiało spełniać rygorystyczne wymagania dotyczące niskich opóźnień oraz niezawodności w zapisie strumieni danych biometrycznych. W kolejnych podrozdziałach szczegółowo omówiono architekturę systemu klient-serwer, zdefiniowano przepływ informacji, uzasadniono dobór technologii oraz przedstawiono zaplanowane scenariusze eksperymentalne.

### 3.1. Architektura testbed'u

Zaprojektowane środowisko eksperymentalne opiera się na rozproszonej architekturze klient-serwer. Struktura systemu została zoptymalizowana pod kątem minimalizacji opóźnień w transmisji strumieniowej danych behawioralnych oraz zapewnienia pełnej

separacji pomiędzy warstwą akwizycji, logiką serwera a chmurową bazą danych. Szczegółowy schemat powiązań komponentów oraz kierunki przepływu informacji w systemie przedstawiono na rysunku 3.1.



Rysunek 3.1. Schemat architektury systemu

### 3.1.1. Komponenty systemu

W skład środowiska laboratoryjnego wchodzi trzy główne węzły technologiczne:

- **Klient (Aplikacja Webowa):** Warstwa prezentacji uruchamiana bezpośrednio w przeglądarce internetowej użytkownika. Skrypt napisany w języku JavaScript odpowiada za obsługę interfejsu gry oraz ciągłe, asynchroniczne przechwytywanie surowych współrzędnych kursora myszy ( $x, y$ ) oraz jej stan wraz z powiązanymi znacznikami czasu (*timestamp*) oraz aktualnym stanem punktowym (*score*).
- **Serwer Aplikacyjny (Render.com):** Centralny punkt systemu zrealizowany w języku Python przy użyciu frameworka Flask, hostowany na platformie chmurowej Render.com pod kontrolą produkcyjnego serwera WSGI Gunicorn. Wewnątrz tego komponentu wydzielono cztery logiczne podmoduły odpowiedzialne za przetwarzanie danych:
  - *Aplikacja Flask* – obsługuje routing API, sesje użytkowników oraz dwukierunkową komunikację;
  - *Moduł normalizacji* – odpowiada za proces inżynierii cech (*feature engineering*), transformując surowe punkty  $(x, y, t)$  na wektory cech biometrycznych (m.in. prędkość, odległość od celu);
  - *Moduł decyzyjny* – dokonuje klasyfikacji i oceny autentyczności ruchu w czasie rzeczywistym przy użyciu modelu Random Forest;
  - *Moduł treningowy* – zarządza procedurami uczenia maszynowego oraz aktualizacją wag klasyfikatora.
- **Baza Danych (MongoDB Atlas):** Chmurowa, dokumentowa baza danych NoSQL dostarczana w modelu SaaS. Odpowiada za bezpieczne przechowywanie logów w formie strukturyzowanych dokumentów JSON w kolekcji *sesje\_uzytkownikow*.

#### 3.1.2. Przepływ danych (Data Flow)

Przepływ informacji pomiędzy poszczególnymi warstwami stanowiska badawczego realizowany jest w ramach trzech głównych etapów (rys. 3.1):

1. **Akumulacja i przesył danych (Klient → Serwer):** W trakcie trwania gry, surowe zdarzenia ruchu myszy są tymczasowo gromadzone w pamięci przeglądarki w tablicy `biometricData`. Przesłanie zgromadzonych danych na serwer następuje wsadowo, w jednym pakiecie, na sam koniec sesji (po wywołaniu funkcji `stopGame`), co pozwala na znaczną redukcję narzutu komunikacyjnego.
2. **Przekazanie zebranych danych (Serwer → Baza Danych):** Po odebraniu pakietu danych z klienta, serwer Flask agreguje zebraną serię zdarzeń i przesyła ją do bazy MongoDB Atlas w celu trwałego zapisu sesji.
3. **Ekstrakcja danych do treningu (Baza Danych → Serwer):** W momencie wywołania procedury uczenia lub aktualizacji profilu biometrycznego, moduł treningowy pobiera z bazy danych historyczne, zbalansowane pakiety sesji wzorcowych danego użytkownika oraz sesje stanowiące tło (anomalie).
4. **Predykcja i weryfikacja (Serwer → Klient):** Moduł decyzyjny serwera ekstrahuje cechy z odebranych zdarzeń i poddaje je klasyfikacji. Jeśli uśredniona pewność modelu (`confidence`) wynosi minimum 50%, serwer autoryzuje sesję i przesyła decyzję o przyznaniu dostępu do przeglądarki.

#### 3.2. Wykorzystane technologie

Aby zrealizować założenia projektowe, wybrano nowoczesny stos technologiczny, który gwarantuje wysoką wydajność oraz łatwość skalowania:

- **Python (Flask & Gunicorn):** Język Python wybrano ze względu na natywne wsparcie dla bibliotek sztucznej inteligencji. Mikroframework Flask posłużył do szybkiego budowania API, obsługi punktów końcowych (`endpoints`) oraz zarządzania routinguem aplikacji, natomiast Gunicorn zapewnił współbieżną obsługę żądań w środowisku produkcyjnym.
- **WebSockets (Flask-SocketIO):** Zastosowano w celu utrzymania stałego, dwukierunkowego połączenia między klientem a serwerem. Jest to technologia kluczowa dla przesyłania setek koordynatów myszy na sekundę bez opóźnień charakterystycznych dla klasycznych żądań HTTP REST.
- **MongoDB Atlas & PyMongo:** Wybór nierzelacyjnej bazy danych NoSQL podyktowany był charakterystyką gromadzonych logów. Zdarzenia biometryczne mają postać szeregów czasowych o zmiennej długości, z czym format dokumentowy (JSON) radzi sobie znacznie lepiej niż klasyczne tabele SQL. Biblioteka PyMongo (klasa `MongoClient`) posłużyła jako sterownik do bezpośredniej integracji i komunikacji backendu z bazą danych.



- **Render:** Platforma chmurowa działająca w modelu PaaS (*Platform as a Service*), wykorzystana do stabilnego hostowania i produkcyjnego uruchomienia serwera aplikacyjnego. Zapewnia ona automatyczne wdrażanie kodu z repozytorium oraz bezproblemową integrację ze środowiskiem uruchomieniowym Pythona.
- **Pandas & NumPy:** Standardowe, wysoce zoptymalizowane biblioteki w ekosystemie Pythona. Biblioteka Pandas posłużyła do strukturyzacji danych, mapowania cech i tworzenia ramek danych (*DataFrame*), natomiast NumPy wykorzystano do szybkich operacji macierzowych, przekształceń wymiarów danych oraz obliczeń matematycznych.
- **Scikit-Learn (Zastosowanie docelowe):** Pakiet wykorzystany do implementacji docelowego algorytmu klasyfikacyjnego *Random Forest* (*RandomForestClassifier*). Użyty również do preprocessingu danych: standaryzacji wektorów cech (*StandardScaler*), podziału danych na zbiór treningowy i testowy (*train\_test\_split*) oraz wyliczania metryk i macierzy pomyłek (*confusion\_matrix*).
- **Joblib:** Biblioteka użyta do zaawansowanej serializacji i deserializacji obiektów języka Python. Wykorzystano ją do trwałego zapisywania wytrenowanych modeli oraz skalerów do plików binarnych *.pkl*, a także ich późniejszego bezpiecznego ładowania podczas fazy inferencji decyzyjnej.
- **Python-dotenv:** Narzędzie (*load\_dotenv*) zastosowane do bezpiecznego zarządzania konfiguracją i sekretami aplikacji. Pozwala na bezproblemowe ładowanie danych uwierzytelniających (np. tajnego klucza URI połączenia z MongoDB) bezpośrednio z lokalnego pliku konfiguracyjnego *.env*.
- **Matplotlib & Seaborn:** Biblioteki graficzne wykorzystane w skryptach testowych algorytmów do generowania wykresów ewaluacyjnych. Umożliwiły automatyczne mapowanie oraz zapisywanie macierzy pomyłek w postaci czytelnych map ciepła (*heatmaps*) do katalogu z wynikami.

#### Technologie wykorzystane wyłącznie w ramach testowania i ewaluacji modeli odrzuconych:

- **TensorFlow & Keras (Modele odrzucone):** Zaawansowany ekosystem głębokiego uczenia maszynowego. Wykorzystany do budowy sieci hybrydowej CNN-LSTM przy użyciu modelu *Sequential* oraz specjalistycznych warstw: konwolucyjnych (*Conv1D*), rekurencyjnych (*Bidirectional*, *LSTM*), warstw regularyzacji (*Dropout*), warstw gęstych (*Dense*) oraz optymalizatora *Adam*.
- **Scikit-Learn - Isolation Forest (Modele odrzucone):** Moduł użyty do zaimplementowania nienadzorowanego algorytmu wykrywania anomalii (*IsolationForest*) w celu zbadania jego skuteczności odizolowania profilu behawioralnego oraz wyliczenia wskaźnika dokładności (*accuracy\_score*) na poziomie pojedynczych zdarzeń.

### 3.3. Scenariusze eksperymentalne

W celu rzetelnej oceny wydajności i bezpieczeństwa zaproponowanego systemu biometrycznego, zdefiniowano następujące scenariusze testowe:

1. **Przebadanie i porównanie modeli klasyfikacyjnych:** Scenariusz polegający na zestawieniu skuteczności wykrywania anomalii przez różne algorytmy uczenia maszynowego. W analizie uwzględniono:
  - *Random Forest* ;
  - *Isolation Forest*;
  - *Sieć neuronowa*;
2. **Ocena ilości danych niezbędnych do ewaluacji:** Eksperyment mający na celu zbadać wpływ liczby zgromadzonych trajektorii na miarodajność weryfikacji tożsamości. Badanie polega na bezpośrednim porównaniu wyników klasyfikacji opartej wyłącznie na pojedynczym, wyizolowanym ruchu ze skutecznością zagregowanej oceny dla pełnej sesji, składającej się z 10 następujących po sobie ruchów.

## 4. Generacja i przygotowanie danych

W procesie budowy systemu opartego na uczeniu maszynowym, jakość, reprezentatywność oraz odpowiednie przygotowanie zbioru uczącego są równie ważne co sam algorytm klasyfikujący. Niniejszy rozdział opisuje metodykę pozyskiwania danych, strukturę zapisanych informacji oraz podejście do analizy i kontroli jakości zebranego materiału.

### 4.1. Proces generacji i zbierania danych

Zbiór danych został wygenerowany z wykorzystaniem opisanego wcześniej środowiska badawczego. Autorska aplikacja webowa została wdrożona na platformie chmurowej Render i udostępniona uczestnikom pod adresem <https://mousebiometrics.onrender.com/>. Zadaniem osób biorących udział w eksperymencie było operowanie kursorem myszy w celu wybierania losowo pojawiających się na ekranie obiektów. Aby uniknąć fragmentacji bazy danych i wysyłania niepełnych logów, zaimplementowano buforowanie po stronie klienta. Skrypt BiometricTracker transmitował zgromadzony pakiet danych do serwera wsadowo, wyłącznie po pomyślnym zakończeniu całej sesji, na którą składało się poprawne kliknięcie 10 celów. Następnie zebrane informacje były trwale zapisywane w nierelacyjnej bazie danych MongoDB.

Zaprojektowany system ma charakter uniwersalny, jednakże na potrzeby ewaluacji w niniejszej pracy wyznaczono jednego użytkownika testowego. W eksperymencie wzięło udział 7 unikalnych osób, co pozwoliło na zgromadzenie 116 kompletnych sesji badawczych. Zbiór ten podzielono na dwie klasy:

- **Profil referencyjny (klasa pozytywna):** Zgromadzono 71 sesji pochodzących od

docelowego użytkownika (oznaczonego identyfikatorem `filipp`). Użytkownik ten został wybrany jako profil referencyjny ze względu na największą ilość dostarczonych próbek, co stanowi idealny fundament do zbudowania stabilnego i wiarygodnego wzorca behawioralnego dla modelu.

- **Tło analityczne (klasa negatywna):** Pozostałe 45 sesji wygenerowało 6 osób trzecich. Sesje te pełnią rolę anomalii (prób nieautoryzowanego dostępu). System pobierania danych dąży do zbalansowania zbioru względem klasy pozytywnej; w przypadku niewystarczającej liczby próbek negatywnych, wykorzystywana jest cała dostępna pula.

## 4.2. Opis zebranych danych

Pojedynczy dokument zapisany w kolekcji `sesje_uzytkownikow` w chmurowej bazie danych MongoDB reprezentuje pełną, ukończoną sesję badawczą. Strukturę surowego zrzutu z bazy przedstawia Listing 1.

**Listing 1.** Struktura dokumentu z kolekcji `sesje_uzytkownikow`

```
{
  "_id": "69ff496a9040d3852f03ae03",
  "username": "filipp",
  "events": [
    {
      "type": "mousemove",
      "x": 890,
      "y": 819,
      "timestamp": 1778338145939,
      "diamondLeft": 1133,
      "diamondTop": 530,
      "score": 0
    },
    // ... kolejne obiekty trajektorii az do osiagniecia score: 10
  ]
}
```

Tablica `events` stanowi chronologiczny zapis aktywności. Każdy obiekt posiada pole `type` (np. `mousemove`), które precyzyjnie rejestruje stan akcji. Ponadto wektor zawiera pozycję kursora ( $x, y$ ), znacznik czasu w milisekundach (`timestamp`), współrzędne celu (`diamondLeft`, `diamondTop`) oraz aktualny stan punktowy (`score`). Parametr `score` jest kluczowy – na jego podstawie surowy strumień danych dzielony jest na niezależne trajektorie ruchu.

### 4.3. Analiza jakości danych

Wysoka jakość zgromadzonych danych jest w pierwszej kolejności gwarantowana przez samą architekturę zbudowanej aplikacji webowej. Zaimplementowany w kodzie JavaScript mechanizm sterujący logiką gry sprawia, że dane w tablicy `biometricData` są transmitowane do API serwera wyłącznie po pomyślnym osiągnięciu zdefiniowanego progu punktowego (wywołanie funkcji `stopGame(true)` po dziesięciu poprawnych interakcjach). Dzięki temu rozwiązaniu odgórnie wyeliminowano problem fragmentacji bazy danych oraz zapisywania przerwanych lub niekompletnych sesji pomiarowych.

Kluczowym założeniem biometrii behawioralnej jest fakt, że każdy użytkownik może operować urządzeniem wskazującym w sposób wysoce nieprzewidywalny i unikalny. Aby zapewnić miarodajność zebranych danych, każdego z uczestników badania poproszono o jak najbardziej naturalne zachowanie. Polegało ono na bezpośrednim zmierzaniu kursorem do celu i unikaniu celowego generowania sztucznych anomalii, na przykład intencjonalnego kręcenia myszką w kółko. Z tego względu w projekcie przyjęto strategię nieodrzućcia żadnych zarejestrowanych sesji na podstawie rzekomo „nietypowego” lub „zbyt chaotycznego” kształtu trajektorii. Wszelkie odstępstwa od idealnego toru ruchu, przestoje czy wahania kursora nie stanowią szumu do usunięcia, lecz są integralną częścią indywidualnego wzorca motorycznego użytkownika, niosącą istotną wartość decyzyjną dla modeli uczących.

## 5. Feature engineering i preprocessing

Surowe strumienie zdarzeń myszy zebrane podczas interakcji nie nadają się bezpośrednio do zasilenia modeli uczenia maszynowego. W tym rozdziale omówiono proces transformacji tych danych w użyteczne wektory cech behawioralnych realizowany przez zaimplementowany moduł normalizacyjny.

### 5.1. Okna czasowe i agregacja danych

Zgodnie z implementacją algorytmu, dane z surowego strumienia dzielone są na logiczne okna czasowe reprezentujące pojedyncze, niezależne trajektorie ruchu. Proces ten opiera się na atrybucie `score`. Zgrupowanie zdarzeń posiadających identyczną wartość punktową pozwala na wyizolowanie pełnego ruchu – od momentu pojawienia się celu na ekranie, aż do jego ostatecznego kliknięcia. Dla każdego tak wyodrębnionego okna wyliczany jest jeden, spójny wektor cech.

### 5.2. Opis wyekstrahowanych cech

Z każdej poprawnej trajektorii ekstrahowany jest zestaw siedmiu metryk:

- `duration_ms`: Całkowity czas trwania ruchu w oknie (różnica wartości `timestamp` między ostatnim a pierwszym zdarzeniem).

- **ideal\_distance**: Najkrótsza teoretyczna droga do celu – odległość euklidesowa w linii prostej między początkowym położeniem kursora ( $x$ ,  $y$ ) a matematycznym środkiem obiektu docelowego (`diamondLeft`, `diamondTop`).
- **actual\_distance**: Rzeczywista długość przebytej ścieżki, obliczana jako suma dystansów euklidesowych między wszystkimi kolejnymi punktami ( $x$ ,  $y$ ) składowymi trajektorii.
- **path\_efficiency**: Wskaźnik efektywności trasy, definiowany jako stosunek `ideal_distance` do `actual_distance`.
- **avg\_speed**: Średnia prędkość operowania myszą, wyrażona jako stosunek `actual_distance` do `duration_ms`.
- **points\_recorded**: Suma zarejestrowanych zdarzeń w danej trajektorii.
- **click\_error\_distance**: Precyzja ostatecznego najechania na cel, mierzona jako odległość euklidesowa między współrzędnymi ( $x$ ,  $y$ ) ostatniego zdarzenia w trajektorii a fizycznym środkiem diamentu (`diamondLeft`, `diamondTop`).

### 5.3. Uzasadnienie wyboru cech

Wybór powyższych metryk opiera się na fundamentach biometrii behawioralnej, z założeniem, że koordynacja oko-ręka oraz dynamika motoryczna są unikalne dla każdego człowieka.

- **Dynamika i czas** (`duration_ms`, `avg_speed`, `points_recorded`): Pozwalają na odróżnienie użytkowników o profilu impulsywnym od ostrożnych. Zmienne te są dodatkowo silnie skorelowane ze sprzętem użytkownika (np. osobiste preferencje czułości DPI oraz częstotliwość raportowania myszy).
- **Kształt i efektywność trasy** (`actual_distance`, `path_efficiency`): Ujawniają podświadome nawyki, takie jak wahania, mikrokorekty toru, „wymachiwanie” kursorem czy naturalne drżenie dłoni. Odstępstwo dystansu rzeczywistego od idealnego precyzyjnie obrazuje płynność ruchu.
- **Precyzja końcowa** (`click_error_distance`): Definiuje indywidualną tolerancję na niedokładność podczas klikania (zjawiska *overshooting* oraz *undershooting*), co jest wysoce powtarzalną cechą przypisaną do konkretnej osoby.

### 5.4. Transformacje danych (normalizacja i filtracja)

Przekształcanie danych surowych w gotowe datasety treningowe wymaga zastosowania mechanizmów chroniących przed anomaliami matematycznymi i sprzętowymi. W module `BiometricNormalizer` zaimplementowano następujące filtry, dokładnie odzwierciedlające logikę przetwarzania:

- **Odrzucanie pustych trajektorii**: Elementy niezawierające prawidłowych współrzędnych przestrzennych (gdzie wszystkie pozycje są wartościami typu `Null`) są automatycznie odrzucane z procesu agregacji.

- **Ochrona przed anomalią dzielenia przez zero:** Ze względu na precyzję systemowych znaczników czasu, przeglądarka może zareportować kilka zdarzeń w ciągu tej samej milisekundy. Aby zapobiec błędom podczas wyliczania prędkości, czas trwania wymuszany jest na poziomie minimum 1 milisekundy. Analogicznie, wskaźniki złożone (jak efektywność ścieżki i prędkość średnia) wyliczane są wyłącznie pod warunkiem, że rzeczywisty dystans (`actual_distance`) jest ściśle większy od zera.
- **Etykietowanie klas i przetwarzanie zbioru:** Na etapie przygotowywania ostatecznej ramki danych (`DataFrame`), do wektorów dodawana jest binarna flaga `is_target_user`. Profil referencyjny otrzymuje klasę pozytywną (1), a anomalie klasę negatywną (0). Następnie cały połączony zbiór poddawany jest wymuszonemu losowemu przetwarzaniu wszystkich wierszy, co stanowi krytyczny element zapobiegający sztucznemu dopasowaniu modelu na podstawie sekwencji historycznej sesji.

## 6. Analiza, uczenie i ewaluacja modeli uczenia maszynowego

Zgromadzone i znormalizowane wektory cech posłużyły do budowy, optymalizacji oraz ewaluacji algorytmów klasyfikujących. Zadaniem zaimplementowanych modeli jest jednoznaczne rozróżnienie prawowitego właściciela konta od intruza na podstawie dynamiki ruchu myszy. Ewaluację skuteczności przeprowadzono z wykorzystaniem macierzy pomyłek oraz fundamentalnych metryk biometrycznych: FAR (odsetek błędnych akceptacji) i FRR (odsetek błędnych odrzuceń).

### 6.1. Konfiguracja eksperymentów

Zbiór wektorów cech, zbalansowany i oetykietowany w module preprocessingu, został poddany standardowej procedurze ewaluacji maszynowej.

- **Podział zbioru:** Dane zostały podzielone na zbiór treningowy i testowy w proporcji 80/20, przy zachowaniu stratyfikacji (utrzymaniu równego rozkładu klas autentycznych i anomalii w obu podzbiorach).
- **Cross-validation:** Aby zapewnić rzetelność wyników i zminimalizować ryzyko przetrenowania (*overfittingu*) modelu Random Forest, proces uczenia wspomagano 5-krotną walidacją krzyżową (5-fold cross-validation).
- **Poziomy oceny:** Ewaluację przeprowadzono w dwóch trybach. Pierwszym z nich była ocena pojedynczej, wyizolowanej trajektorii ruchu (Pojedynczy Event). Drugim była agregacja wyników z pełnej sesji (10 następujących po sobie zdarzeń), gdzie ostateczny werdykt weryfikacyjny był podejmowany na podstawie głosowania większościowego (Cała Sesja).
- **Strategia balansowania klas:** Procedura przygotowania danych wejściowych dąży do zrównoważenia zbioru w proporcji 50/50. W przypadku naturalnego deficytu danych

po stronie klas negatywnych, system automatycznie pobiera pełną dostępną pulę anomalii bez sztucznego generowania syntetycznych próbek. W opisywanym środowisku eksperymentalnym przełożyło się to na wykorzystanie 71 sesji profilu referencyjnego oraz wszystkich 45 dostępnych sesji intruzów, co ukształtowało ostateczny rozkład klas na asymetrycznym poziomie około 60/40 na korzyść klasy pozytywnej.

### 6.2. Wybór i selekcja algorytmów klasyfikacyjnych

W ramach prac badawczych przetestowano trzy zróżnicowane podejścia algorytmiczne: uczenie nienadzorowane (Isolation Forest), głębokie sieci neuronowe (CNN-LSTM) oraz modele zespołowe (Random Forest).

#### 6.2.1. Modele odrzucone i ich wyniki

Przed podjęciem decyzji o wyborze ostatecznej architektury klasyfikatora, przeprowadzono szerokie testy porównawcze z alternatywnymi modelami uczenia maszynowego. Analizie poddano zarówno nienadzorowany algorytm dedykowany do detekcji anomalii, jak i zaawansowaną sieć głęboką. Pomimo odmiennych założeń teoretycznych i różnych poziomów skuteczności, oba te podejścia okazały się nieoptymalne z punktu widzenia specyfiki systemu produkcyjnego działającego w tle przeglądarki. Poniżej przedstawiono szczegółową metodykę oraz bezpośrednie przyczyny dyskwalifikacji poszczególnych rozwiązań.

##### 6.2.1.1. Isolation Forest - metodyka i parametry modelu

W ramach eksperymentów przetestowano algorytm Isolation Forest, służący do nienadzorowanego wykrywania anomalii. Proces przygotowania danych i ewaluacji opierał się na następującej metodyce:

- **Podział danych:** Zastosowano podział na zbiór treningowy i testowy w proporcji 70% do 30% (`test_size=0.3`).
- **Skalowanie:** Wektory cech ustandaryzowano (`StandardScaler`), aby różnice w rzędach wielkości poszczególnych metryk nie zaburzały obliczania dystansów wielowymiarowych.
- **Proces decyzyjny:** Autentyczność weryfikowano dla całych sesji metodą głosowania większościowego, gdzie autoryzacja wymagała średniej predykcji  $\geq 0.5$  dla zagregowanego pakietu zdarzeń.

Hiperparametry modelu klasyfikującego:

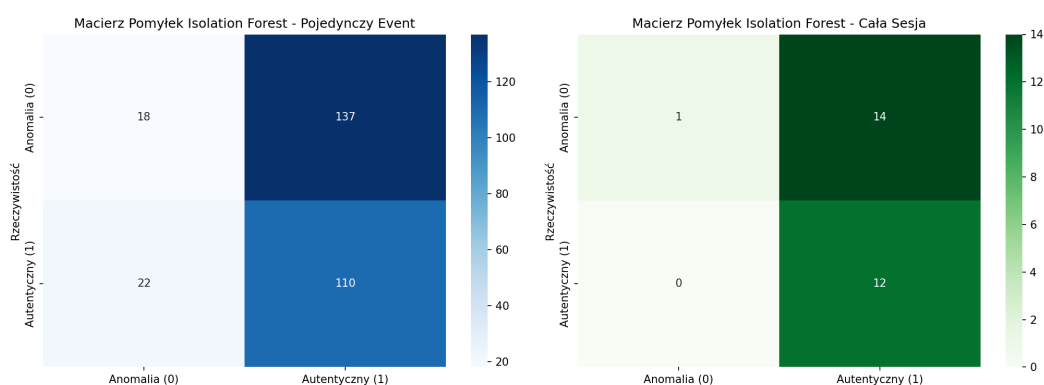
- `n_estimators=100`: Zespół 100 drzew izolujących, co jest optymalną wartością dla 7-wymiarowej przestrzeni cech.
- `contamination=0.4`: Odsetek anomalii ustalono na poziomie 40%, co odzwierciedla faktyczny udział intruzów w zgromadzonym ogólnym zbiorze badawczym (45 z 116 wszystkich sesji).

- `random_state=99`: Stałe ziarno losowości gwarantujące powtarzalność trenowania.

### Powód odrzucenia modelu

Mimo optymalizacji hiperparametrów i dostosowania parametru zanieczyszczenia (contamination) do rzeczywistego rozkładu danych, model Isolation Forest został odrzucony jako mechanizm zabezpieczający z następujących powodów:

1. **Skrajnie wysoki wskaźnik błędnych akceptacji (FAR):** Jak ukazuje lewa macierz pomyłek dla pojedynczych zdarzeń (Rysunek 6.1), model dopuścił aż 137 błędnych akceptacji intruza, poprawnie klasyfikując zaledwie 18 anomalii. Taka proporcja całkowicie dyskwalifikuje algorytm w kontekście cyberbezpieczeństwa.
2. **Porażka w ujęciu zagregowanym (Cała Sesja):** Prawa część wspomnianego wykresu (Rysunek 6.1) dowodzi, że mechanizm sesyjnego głosowania większościowego nie naprawił błędów klasyfikatora. Przepuszczenie 14 z 15 nieautoryzowanych sesji oznacza niemal stuprocentową nieskuteczność.
3. **Niedopasowanie koncepcyjne algorytmu:** Model Isolation Forest zakłada, że anomalie są wyraźnie oddzielone od autentycznych danych. Naturalna, codzienna zmienność motoryczna prawowitego użytkownika jest jednak na tyle duża, że system nie potrafi wyznaczyć ścisłej granicy separującej jego profil od cech osób trzecich, co skutkuje masowym akceptowaniem intruzów (Rysunek 6.1).
4. **Problem z separacją danych przy rozkładzie 60/40:** Chociaż wymuszono na modelu odrzucanie 40% najdalszych próbek, algorytm nadal nie potrafił poprawnie wskazać anomalii. Dowodzi to, że nienadzorowana izolacja matematyczna nie sprawdza się przy tej konkretnej przestrzeni cech behawioralnych.



**Rysunek 6.1.** Macierz pomyłek dla modelu Isolation Forest. Widoczna niemal całkowita nieskuteczność w odrzucaniu anomalii (klasy 0).

### 6.2.1.2. Hybrydowa sieć CNN-LSTM - metodyka i architektura modelu



W ramach eksperymentów przetestowano zaawansowany model głębokiego uczenia maszynowego, łączący konwolucyjne sieci neuronowe (CNN) z rekurencyjnymi komórkami pamięci (LSTM). Proces przygotowania danych opierał się na następujących krokach:

- **Podział danych:** Zastosowano standardowy podział na zbiór treningowy (70%) i testowy (30%) przy użyciu stałego ziarna losowości (`random_state=99`). Ponadto 20% zbioru treningowego wydzielono jako zbiór walidacyjny wewnątrz procesu uczenia (`validation_split=0.2`).
- **Transformacja przestrzeni wejściowej:** Po ustandaryzowaniu metryk (`StandardScaler`), wektory cech zostały matematycznie przekształcone do postaci trójwymiarowej macierzy (`reshape`). Jest to strukturalny wymóg wejściowy dla jednowymiarowych warstw konwolucyjnych. Etykiety klas poddano kodowaniu kategorialnemu (`one-hot encoding`).
- **Proces decyzyjny:** Autentyczność całej sesji weryfikowano poprzez agregację predykcji poszczególnych zdarzeń. Jeśli średnie prawdopodobieństwo wyjściowe wynosiło  $\geq 0.5$ , sesję autoryzowano.

Ze względu na specyfikę danych biometrycznych, zaprojektowano następującą architekturę sieci i hiperparametry:

- **Ekstrakcja cech (CNN):** Dwie warstwy `Conv1D` (64 i 32 filtry, funkcja aktywacji `ReLU`) odpowiadające za wydobycie ukrytych wzorców i lokalnych zależności przestrzennych z wektora danych.
- **Analiza sekwencyjna (LSTM):** Warstwa dwukierunkowa `Bidirectional` (LSTM) oraz standardowa LSTM. Mechanizmy te pozwalają sieci zapamiętać chronologiczny kontekst zachowań motorycznych użytkownika.
- **Klasyfikacja (Dense):** Zespół w pełni połączonych warstw gęstych (128 i 64 neurony), zakończony warstwą wyjściową z funkcją `softmax` dystrybuującą prawdopodobieństwo na dwie klasy (autentyczny lub intruz).
- **Parametry uczenia:** Model trenowano przez 50 epok z rozmiarem wsadu (`batch_size`) równym 16. Zastosowano optymalizator Adam ze współczynnikiem uczenia na poziomie 0.001.

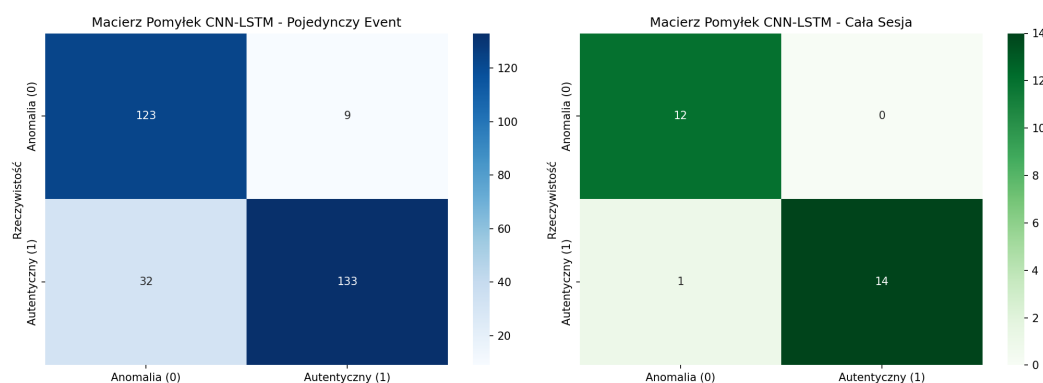
### Powód odrzucenia modelu pomimo wysokiej skuteczności

Hybrydowa sieć CNN-LSTM wykazała się bardzo wysoką skutecznością predykcyjną. Na poziomie całej sesji algorytm osiągnął zerowy wskaźnik FAR (wykrył wszystkie 12 anomalii) oraz FRR na poziomie zaledwie 6,6% (1 błędne odrzucenie), co ilustruje Rysunek 6.2. Mimo tak znakomych rezultatów i niemal bezbłędnej macierzy pomyłek, model ten został ostatecznie odrzucony jako docelowe rozwiązanie z następujących powodów:

1. **Bariera wydajnościowa i sprzętowa:** Zaprojektowana architektura, ze względu na obecność wielu warstw konwolucyjnych, rekurencyjnych (LSTM) i gęstych, generuje

ogromną liczbę trenowalnych parametrów (wag). Wymaga to znacznych zasobów obliczeniowych (CPU/GPU) podczas inferencji.

2. **Nieadekwatność do środowiska Web:** Podstawowym celem projektu jest system działający w tle przeglądarki. Uruchamianie tak złożonego i obciążającego pamięć modelu głębokiego w środowisku klienta (np. za pomocą JavaScript) drastycznie obniżyłoby wydajność stacji roboczej prawowitego użytkownika, wpływając negatywnie na płynność korzystania z systemu.
3. **Problem czasu rzeczywistego (Latency):** Skomplikowane operacje macierzowe wprowadzają zauważalne opóźnienia w klasyfikacji. W kontekście ciągłego uwierzytelniania behawioralnego, decyzje muszą być podejmowane w ułamkach sekund bez zawieszania głównego wątku aplikacji.



**Rysunek 6.2.** Macierz pomyłek dla modelu CNN-LSTM w ujęciu pojedynczego zdarzenia oraz całej sesji

### 6.2.2. Wybrany model docelowy i jego wynik (Random Forest)

Klasyfikator **Random Forest** został ostatecznie wybrany jako docelowy model systemu. Algorytm ten osiągnął najlepsze wyniki predykcyjne spośród wszystkich testowanych struktur (całkowita bezbłądność w ujęciu sesyjnym), a jednocześnie jego architektura jest znacznie lżejsza pod kątem wymagań obliczeniowych niż sieć CNN-LSTM, co zadecydowało o wdrożeniu go do dalszych prac.

Proces przygotowania danych oraz konfiguracja modelu opierały się na zaktualizowanej metodzie:

- **Podział i transformacja:** Unikalne sesje podzielono na zbiór treningowy (70%) i testowy (30%) przy zachowaniu stratyfikacji (`test_size=0.3`). Następnie wektory cech poddano standaryzacji (`StandardScaler`).
- **Proces decyzyjny (Inference):** Wykorzystano zagregowaną metodę weryfikacji. Model ocenia każdy wyekstrahowany ruch niezależnie, a ostateczny werdykt o autentyczności sesji zapada, gdy średnia predykcji wszystkich zdarzeń w oknie wynosi co najmniej 0.5 (`confidence = np.mean(predictions) >= 0.5`).

Model lasu losowego został zainicjalizowany w podstawowej, stabilnej konfiguracji:

- `n_estimators=100`: Zespół zbudowany ze 100 niezależnych drzew decyzyjnych, co zapewnia wysoką dokładność przy zoptymalizowanym czasie treningu i predykcji.
- `random_state=99`: Stałe ziarno losowości gwarantujące pełny determinizm i powtarzalność procesu trenowania w środowisku testowym.

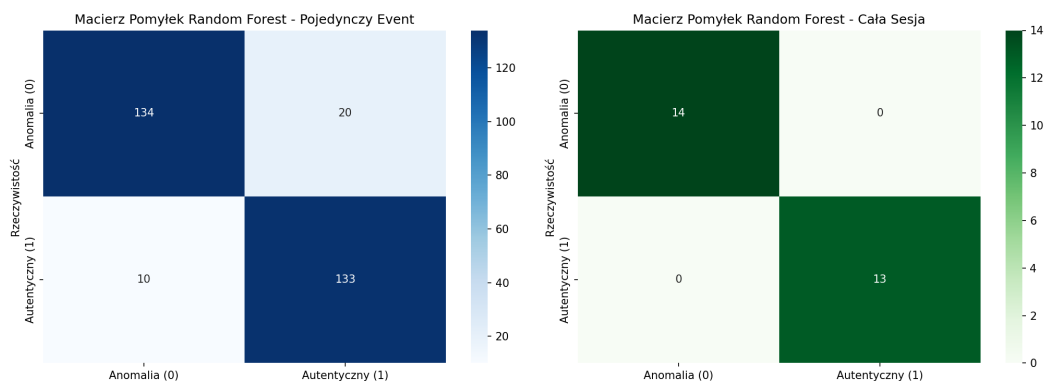
### Wyniki ewaluacji i szczegółowe uzasadnienie wyboru

Na poziomie pojedynczego zdarzenia Random Forest wykazał się doskonałym balansem, poprawnie klasyfikując 134 anomalie oraz 133 zdarzenia autentyczne, przy zaledwie 20 błędnych akceptacjach i 10 błędnych odrzuceniach.

W docelowym trybie weryfikacji na poziomie zagregowanej sesji (10 ruchów) algorytm całkowicie wyeliminował błędy: poprawnie zidentyfikował 14 na 14 sesji intruzów (FAR = 0%) oraz 13 na 13 sesji prawowitego użytkownika (FRR = 0%).

Decyzja o wyborze tego klasyfikatora do implementacji docelowej w klasie `BiometricDecision` została podyktowana następującymi względami technicznymi:

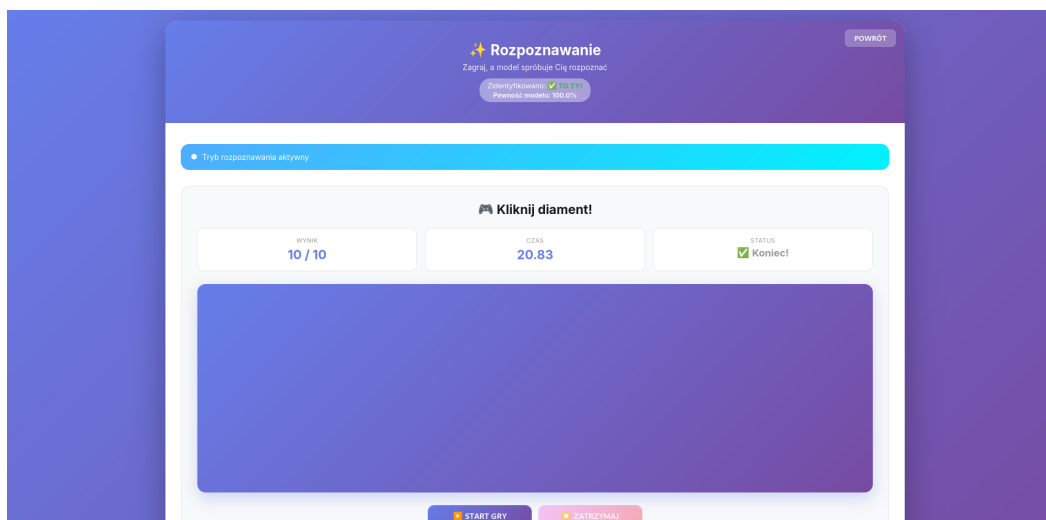
1. **Zoptymalizowany koszt obliczeniowy:** Mimo swojej złożoności, architektura lasu losowego opartego na 100 drzewach jest nieporównywalnie lżejsza w fazie inferencji niż głębokie sieci neuronowe (CNN-LSTM). Oferuje błyskawiczne wnioskowanie, co jest kluczowe dla zachowania płynności działania aplikacji webowej w czasie rzeczywistym, nie obciążając przeglądarki użytkownika.
2. **Najwyższa skuteczność (Bezблędność sesyjna):** Mechanizm uśredniania pewności całkowicie wygładził pojedyncze potknięcia algorytmu widoczne na poziomie zdarzeń. Pozwoliło to na deklasację modelu Isolation Forest i osiągnięcie stabilności dorównującej sieciom głębokim (Rysunek 6.3).
3. **Stabilność przy asymetrii danych:** Model poprawnie uogólnił sygnaturę biometryczną prawowitego użytkownika, pomimo braku sztucznego zbalansowania klas do proporcji 50/50 (wynikającego z naturalnego deficytu anomalii w zbiorze). Świadczy to o dużej elastyczności algorytmu i odporności na zakłócenia wynikające z nierównej ilości danych wejściowych.



**Rysunek 6.3.** Macierz pomyłek dla docelowego modelu Random Forest w ujęciu pojedynczego zdarzenia oraz całej sesji

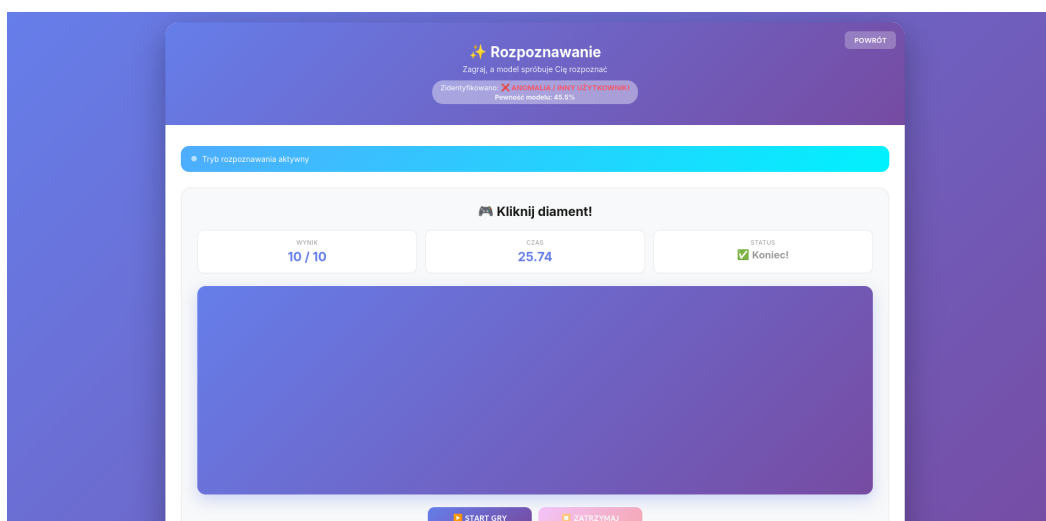
### 6.3. Eksperyment na gotowym środowisku

Działanie wybranego modelu zweryfikowano w warunkach docelowych, implementując mechanizm jako działający w tle skrypt przeglądarki. Poniższe zrzuty ekranu prezentują zachowanie systemu w czasie rzeczywistym.



**Rysunek 6.4.** System poprawnie weryfikuje dynamikę ruchu i pozwala na swobodne korzystanie z konta prawowitemu użytkownikowi.

W sytuacji wykrycia intruza system analizuje na bieżąco dynamikę kursora. Jeśli w ramach zebranej sesji pewność modelu (ang. *confidence*) co do autentyczności użytkownika spada poniżej progu 50%, system natychmiast klasyfikuje ruch jako anomalię i odrzuca dostęp.



**Rysunek 6.5.** Prawidłowe rozpoznanie intruza – pewność modelu klasyfikującego wynosi  $< 50\%$ , co skutkuje odrzuceniem sesji.

## 6.4. Analiza wpływu przygotowania danych na efektywność klasyfikacji

Stuprocentowa skuteczność modelu w ujęciu sesyjnym wynika bezpośrednio z przyjętej metodologii agregacji. Pojedynczy ruch kursora obarczony jest szumem (np. mikroko-  
rektami i chwilowym rozproszeniem), co widać po niewielkich błędach na macierzach pojedynczych zdarzeń. Przetwarzanie i buforowanie kompletnych, 10-punktowych sesji pozwala uśrednić te losowe zakłócenia. Osobista sygnatura biometryczna stabilizuje się w dłuższym oknie obserwacji, dostarczając klasyfikatorowi informacji o wysokiej czystości.

Mimo doskonałych wyników w przeprowadzonym eksperymencie, należy zaznaczyć, że walidacja odbyła się na próbie badawczej o ograniczonej wielkości. Aby zyskać absolutną pewność co do niezawodności i stabilności modelu oraz całkowicie wykluczyć zjawisko przeuczenia na specyficzne wzorce ruchowe grupy testowej, niezbędne byłoby przebadanie systemu na znacznie większej, masowej próbie danych. Pozwoliło to na ostateczne potwierdzenie bezbłędności modelu w bardzo zróżnicowanych warunkach i na wielu typach urządzeń wskazujących.

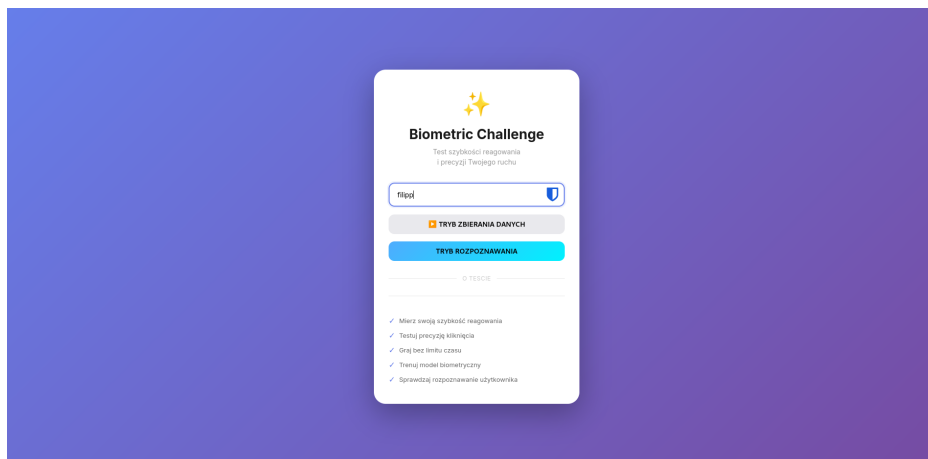
## 7. Działanie aplikacji i scenariusz użytkownika

Zaprojektowana aplikacja webowa *Biometric Challenge* oferuje intuicyjny interfejs graficzny, który przeprowadza badanego przez wszystkie etapy eksperymentu: od rejestracji ruchu, przez trenowanie modelu, aż po ostateczną weryfikację tożsamości. Architektura rozwiązania wymusza ścisły, czteroetapowy przepływ pracy (ang. *user flow*), który został szczegółowo opisany poniżej.

### 7.0.1. Krok 1: Logowanie i wybór trybu pracy

Interakcja z systemem rozpoczyna się na ekranie uwierzytelniania. Użytkownik wprowadza swój unikalny identyfikator (np. *filipp*), który pełni kluczową rolę w architekturze – posłuży do przypisania gromadzonych logów do odpowiedniego dokumentu w bazie MongoDB. Na tym etapie system wymusza wybór jednej z dwóch ścieżek dedykowanych do obsługi interfejsu (Rysunek 7.1):

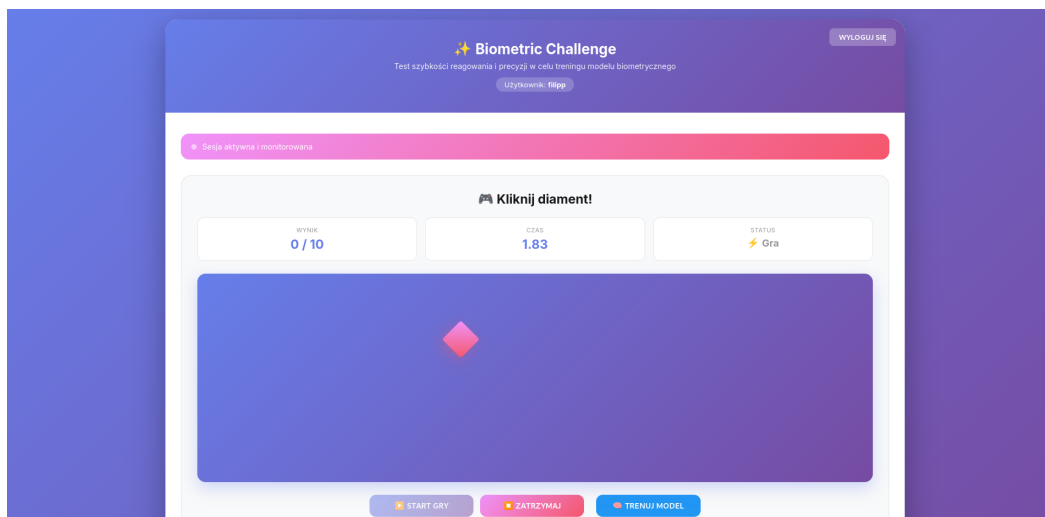
- **Tryb Zbierania Danych** – służący do budowy profilu referencyjnego.
- **Tryb Rozpoznawania** – służący do testowania gotowego klasyfikatora.



Rysunek 7.1. Ekran powitalny aplikacji z wyborem trybu działania

### 7.0.2. Krok 2: Tryb Zbierania Danych

Po wyborze trybu zbierania danych, uczestnik badania przenoszony jest do właściwego interfejsu gry biometrycznej. Jego zadaniem jest precyzyjne nakierowanie kursora i kliknięcie w pojawiający się losowo na ekranie obiekt (diament). Przez cały czas trwania rozgrywki, umieszczony w kodzie strony skrypt `tracker.js` działający w tle przeglądarki asynchronicznie monitoruje i buforuje współrzędne myszy oraz znaczniki czasu. O aktywnym rejestrowaniu danych informuje odpowiedni panel interfejsu (Rysunek 7.2). Aby zapobiec fragmentacji danych w bazie, pakiet logów wysyłany jest do serwera API (ścieżka `/api/biometrics`) dopiero po pomyślnym ukończeniu pełnej sesji, czyli zdobyciu 10 punktów.

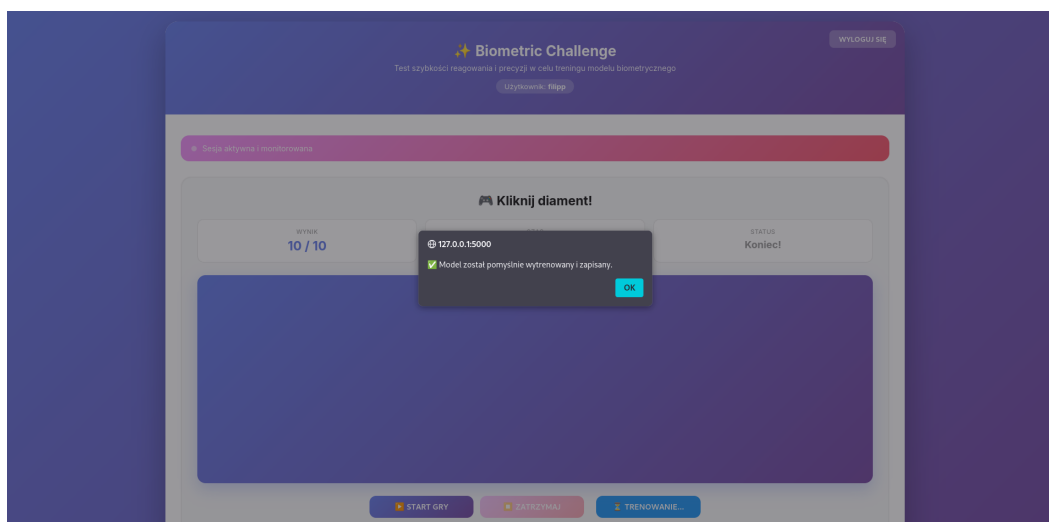


Rysunek 7.2. Interfejs gry w trybie zbierania danych z aktywnym monitorowaniem ruchu

### 7.0.3. Krok 3: Trenowanie modelu profilu

Po zgromadzeniu wystarczającej liczby sesji pozytywnych i negatywnych w bazie danych, użytkownik może zainicjować proces uczenia maszynowego bezpośrednio z

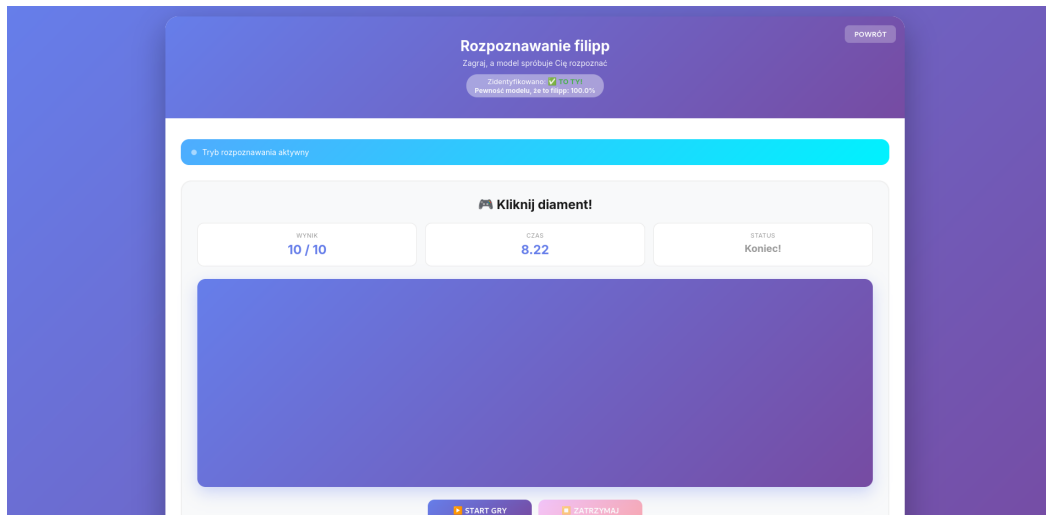
poziomu interfejsu webowego. Po kliknięciu odpowiedniego przycisku, system wykonuje asynchroniczne żądanie do punktu końcowego `/api/train`. Serwer backendowy (klasa `BiometricTrainer`) pobiera zbalansowany zbiór danych, ekstrahuje cechy, a następnie trenuje docelowy model Random Forest. Proces kończy się wygenerowaniem i zapisaniem plików `model_username.pkl` oraz `scaler_username.pkl` w strukturze katalogowej serwera. O pomyślnym zakończeniu operacji użytkownik informowany jest alertem systemowym (Rysunek 7.3).



**Rysunek 7.3.** Potwierdzenie pomyślnego wytrenowania i zapisu modelu klasyfikującego

#### 7.0.4. Krok 4: Tryb Rozpoznawania (Weryfikacja w czasie rzeczywistym)

Gdy spersonalizowany model jest już wygenerowany, użytkownik może logować się do systemu za pomocą opcji "Tryb Rozpoznawania". Procedura gry wygląda identycznie jak w przypadku zbierania logów, jednak tym razem zamknięty pakiet z bufora `tracker.js` trafia do modułu decyzyjnego backendu poprzez ścieżkę `/api/recognize`. Serwer za pomocą klasy `BiometricDecision` ładuje profil użytkownika, ekstrahuje cechy z zebranej trajektorii i weryfikuje jej zgodność ze wzorcem matematycznym (Rysunek 7.4). Jeśli uśredniona predykcja lasu losowego wynosi co najmniej 50%, system autoryzuje sesję i aktualizuje interfejs, wyświetlając komunikat z procentową pewnością dopasowania (np. 100%). W przypadku detekcji anomalii, na ekranie pojawia się odpowiednia informacja.



**Rysunek 7.4.** Pozytywna weryfikacja użytkownika w trybie rozpoznawania wraz z oceną pewności modelu

## 8. Dyskusja i wnioski

Projekt zakończył się obiecującymi wynikami, a wszystkie założone cele badawcze oraz implementacyjne zostały pomyślnie zrealizowane. Docelowy model uczenia maszynowego (Random Forest) osiągnął stuprocentową skuteczność w trybie klasyfikacji całych sesji, redukując wskaźniki FAR (False Acceptance Rate) oraz FRR (False Rejection Rate) do zera. Wyniki eksperymentu jednoznacznie dowodzą, że analiza dynamiki ruchu myszy stanowi wartościową sygnaturę biometryczną, a zaproponowany system może być z powodzeniem wykorzystywany jako działająca w tle, dodatkowa warstwa zabezpieczeń (2FA) w procesie uwierzytelniania.

Mimo obiecujących rezultatów, w ramach dalszego rozwoju projektu warto uwzględnić następujące kierunki badań:

- **Powiększenie zbioru testowego:** Zgromadzenie znacznie większej ilości danych od szerszej grupy użytkowników i ponowne przebadanie modeli, co pozwoli całkowicie wykluczyć zjawisko przeuczenia (overfittingu) i zagwarantuje stabilność systemu w zróżnicowanych warunkach sprzętowych.
- **Wykorzystanie kontekstu aplikacji webowej:** Zintegrowanie analizy biometrycznej z bardziej skomplikowanymi systemami, w których precyzyjnie wiadomo z poziomu przeglądarki, w jaki konkretnie obiekt interfejsu kliknął użytkownik, co pozwoli na głębsze profilowanie kontekstowe.
- **Optymalizacja liczby próbek:** Przeprowadzenie szczegółowych badań w celu określenia, jaka minimalna liczba zarejestrowanych ruchów jest wystarczająca do odpowiednio precyzyjnego i bezpiecznego wykrycia intruza (obecnie system polega na agregacji 10 zdarzeń).



## 8.1. Materiały dodatkowe

### 8.1.1. Repozytorium projektu

Obecnie aplikacja działa wyłącznie w środowisku lokalnym, ponieważ nie została jeszcze wdrożona na zewnętrzny serwer produkcyjny. Pełny kod źródłowy projektu, w tym implementacja backendu Flask, skrypty inżynierii cech, moduły uczenia maszynowego oraz pliki interfejsu użytkownika, znajduje się w publicznym repozytorium w serwisie GitHub pod adresem:

`github.com/FilipPrzygoda/MouseBiometrics-adac`

### 8.1.2. Struktura projektu i opis plików

Architektura rozwiązania została podzielona na logiczne moduły odpowiedzialne za backend (Flask, ML), frontend (HTML/CSS/JS) oraz skrypty badawcze. Poniżej przedstawiono układ najważniejszych plików i katalogów w repozytorium:

- **Katalog główny (/)**

- `app.py` – Główna aplikacja Flask zarządzająca routinguem HTTP, sesjami użytkowników oraz udostępniająca punkty końcowe API (m.in. zbieranie danych, trenowanie, rozpoznawanie).
- `ekstrakcja_cech.py` – Klasa `BiometricFeatureExtractor` transformująca surowe zdarzenia ruchu kursora w ustandaryzowane wektory cech biometrycznych.
- `trening.py` – Moduł (`BiometricTrainer`) integrujący bazę danych z ekstraktorem w celu pobrania danych, wytrenowania i zapisu modelu dla danego użytkownika.
- `decyzja.py` – Moduł (`BiometricDecision`) ładujący wytrenowany model i skaler, odpowiedzialny za proces ewaluacji nowych sesji i podejmowanie ostatecznej decyzji o autentykacji.
- `requirements.txt` – Wykaz zależności środowiska Python (m.in. Flask, scikit-learn, TensorFlow, pymongo).
- `ARCHITEKTURA_MODUŁÓW.md` – Dokumentacja techniczna opisująca przepływ informacji pomiędzy ekstrakcją, trenowaniem a predykcją.

- **Katalog /models** – Przestrzeń zapisu gotowych struktur, zawierająca pliki wytrenowanych klasyfikatorów (`np.model_username.pkl`) oraz skalerów cech (`np.scaler_username.pkl`).

- **Katalog /templates oraz /css** – Widoki aplikacji webowej: `login.html` (ekran uwierzytelniania), `index.html` (panel główny z trybem zbierania danych), `recognition.html` (panel weryfikacji tożsamości) oraz kaskadowe arkusze stylów (`style.css`).

- **Katalog /js** – Logika działania po stronie przeglądarki (frontend):

- `tracker.js` – Kluczowy moduł nasłuchujący i buforujący zdarzenia myszy (po-

zycję, kliknięcia, znaczniki czasu), a następnie wysyłający pakiety danych do API.

- `game.js` – Mechanika logiki testowej (losowe generowanie obiektów, pomiar czasu reakcji, zliczanie poprawnych akcji do limitu sesji).
- `auth.js` – Obsługa sesji uwierzytelniania.
- **Katalog** `/testing-algorithms` i `/testing-results` – Wyizolowane środowisko ewaluacyjne. Zawiera niezależne skrypty badawcze (dla Random Forest, Isolation Forest oraz CNN-LSTM) służące do weryfikacji skuteczności uczenia maszynowego oraz katalog przechowujący wygenerowane macierze pomyłek w formie graficznej.

### 8.1.3. Struktura datasetu

Zgromadzone dane biometryczne są przechowywane w dokumentowej, nierelacyjnej bazie danych MongoDB. Każdy rekord w kolekcji reprezentuje kompletną sesję badawczą (składającą się z 10 poprawnych kliknięć) zapisaną w formacie strukturyzowanym JSON. Architektura pojedynczego dokumentu przedstawia się następująco:

- `_id` – unikalny, automatycznie generowany identyfikator obiektu w bazie danych (np. 69ff496a9040d3852f03ae03),
- `username` – identyfikator tekstowy użytkownika biorącego udział w sesji (np. "filipp"),
- `events` – chronologiczna tablica obiektów rejestrujących surowy ruch myszy. Każdy pojedynczy obiekt w tablicy reprezentuje ułamek sekundy aktywności i zawiera parametry:
  - `type` – typ zarejestrowanego zdarzenia (np. "mousemove"),
  - `x` – współrzędna położenia kursora na osi X (np. 890),
  - `y` – współrzędna położenia kursora na osi Y (np. 819),
  - `timestamp` – dokładny, systemowy znacznik czasu wyrażony w milisekundach (np. 1778338145939),
  - `diamondLeft` – lewa krawędź geometryczna docelowego obiektu (np. 1133),
  - `diamondTop` – górna krawędź geometryczna docelowego obiektu (np. 530),
  - `score` – aktualna liczba punktów zdobyta w grze. Atrybut ten umożliwia algorytmom bezbłędny podział ciągłego strumienia na wyizolowane, niezależne trajektorie pojedynczych ruchów.